LLDD BE - Job 12 NotifyPendingWork

SBP Mall - ระบบประกันรายได้ | Low Level Design Document

แนวทางการใช้เอกสาร

หัวข้อ	คำอธิบาย
วัตถุประสงค์	ใช้เป็นรายละเอียดระดับพัฒนาสำหรับออกแบบ ลงมือพัฒนา ตรวจทาน และทดสอบขอบเขตที่ระบุในฉบับนี้
ลำดับการอ่าน	เริ่มจาก Overview และ Scope จากนั้นตรวจ Field/Validation, Implementation, Contract, Processing Flow, Acceptance Criteria และ Test Checklist ตามลำดับ
ผลลัพธ์ที่คาดหวัง	ผู้อ่านสามารถระบุ input, ขั้นตอนประมวลผล, output, เงื่อนไขผิดพลาด และหลักฐานการทดสอบได้จากเนื้อหาในฉบับเดียว
Java เดิม	ภาคผนวกท้ายฉบับระบุ source file, ช่วงบรรทัด และ code Java เดิมสำหรับตรวจเทียบ behavior ก่อนย้ายระบบ

คำว่า Input, Progress และ Output ในเอกสารนี้หมายถึงข้อมูลตั้งต้น ลำดับการทำงาน และผลลัพธ์ที่ตรวจสอบได้ของขอบเขตที่กำลังพัฒนา

1. บทนำ

1.1 วัตถุประสงค์

เตือนงานค้าง + escalation (ช่วง 30-36 / 45-51 / 60-66 วัน): อ่านงานที่ค้างในขั้นรอดำเนินการของ workflow engine แล้วส่งอีเมลเตือนตามช่วงอายุงาน 30-36 / 45-51 / 60-66 วัน จัดกลุ่มตามโซนของร้านที่ถูกกระทบ ส่งถึง GM และหน่วยงานส่งเสริมธุรกิจ — พอร์ตจากโค้ดเดิม `main/SendMailReport.java` + `service/MailReportService.java` ไม่ใช่งานใหม่ · ช่วง 30-36 / 45-51 / 60-66 วัน มาจาก `FgiConstant.THIRTYSIX/FIFTYONE/SIXTYSIX` ของจริง · ที่แยกเอกสารเมื่อ 2026-09-02 เพราะคลาสนี้ไม่ถูกเขียนไว้ในเอกสาร Batch v4.0

1.2 ขอบเขตงาน

- Main class/script: fgi.main.SendMailReport / FGI_SendMailReport.sh
- Phase: E
- Output: อีเมลเตือนงานค้าง แยกตามโซน 3 ช่วงอายุงาน
- Estimate: 10 ชั่วโมง
- Argument รับผ่าน `INPUT` (JSON) — local ใช้ env `JOB_NAME/INPUT` · AWS Batch ใช้ `argv[3]/argv[2]` · ดูหัวข้อ 5.95 · ไม่มีตาราง job_configs และไม่มีหน้าจอควบคุม (หน้า Flow Batch Job ในกลุ่มเมนู Flow เหลือแค่ Flowchart + Database ที่ใช้ · 2026-08-06)
- ตารางเวลาตั้งที่ AWS Batch scheduled event (repo ไม่มี `@Cron`) — ยกเว้น job ที่เป็น event-driven (ดูหัวข้อ Config Schema ของ job นั้น) ซึ่งห้ามตั้ง schedule · ทุก job ถูกบันทึกลง `integration_log` โดย `main.ts` อัตโนมัติ + structured log `BATCH_START/BATCH_END` พร้อม `runId`

1.3 ข้อมูลส่งมอบ

รายการ	รายละเอียด
Track	BE
Estimate	13 ชั่วโมง = implementation 10 + unit test 3 (30%)
Owner	Aphiwit <Bank> Khammoon
Legacy class เดิม	`SendMailReport` (`fcsJar/src/th/co/gosoft/fgi/main/SendMailReport.java`)
Target repository	`SBP/srm-sps-spsap-sop-sgi-batch` (NestJS 11 + TypeORM · schema `sps_store` · มติ 2026-09-02 — ย้ายมาจาก store-backend) — batch runner ของ SBP ที่รันอยู่แล้ว 42 job บน AWS Batch · ลงทะเบียน job ใน `src/main.ts` แล้วรับ argument ผ่าน `JOB_NAME/INPUT` (local) หรือ `argv[3]/argv[2]` (AWS Batch) · ไม่ผ่าน BFF และไม่เปิด HTTP · ตารางเวลาเป็น AWS Batch scheduled event ไม่ใช่ `@Cron` · ดู `srm sps spsap sop sgi batch`

1.4 เอกสาร LLDD ที่เกี่ยวข้อง

ตารางนี้สร้างจาก endpoint และตารางที่เอกสารฉบับนี้ประกาศไว้จริง — อ่านฉบับที่อยู่ในตารางก่อนลงมือ เพื่อไม่ให้สัญญา request/response หรือชื่อคอลัมน์หลุดจากกัน

ความสัมพันธ์	เอกสาร LLDD	เกี่ยวข้องตรงไหน
สัญญากลาง	LLDD-BE-API-Common-Contracts.pdf	envelope `{success,data}` · error code · pagination · รูปแบบวันที่/เลขเอกสาร
โครงสร้างข้อมูล	LLDD-BE-Database-Structure.pdf	DDL ของตารางที่หัวข้อ Reference DB Mapping อ้างถึง
แพลตฟอร์มระบบเดิม	LLDD-BE-Integration-SBP-Platform.pdf	header จาก BFF (`x-api-key` / `x-user-*`) · การ reuse ตารางและ service ของระบบ SBP เดิม
ต้องจบก่อน (ลำดับงาน)	LLDD-BE-API-Common-Contracts.pdf	เป็นฉบับต้นทางของสัญญา/โครงที่ฉบับนี้อ้าง
ต้องจบก่อน (ลำดับงาน)	LLDD-BE-Job-8b-StartInternalWorkflow.pdf	เป็นฉบับต้นทางของสัญญา/โครงที่ฉบับนี้อ้าง

2. Program Specification

2.1 การลงทะเบียน job และ Arguments (sop-sgi-batch)

งานนี้ลงทะเบียนเป็น job ชื่อ `sgi-notify-pending-work` ใน `src/main.ts` ของ SBP/`srm-sps-spsap-sop-sgi-batch` โดยใช้ dispatcher เดิมของ repo (ไม่สร้าง runner ใหม่) — `main.ts` อ่านชื่อ job และ input มา 2 ทาง แล้วเลือกอัตโนมัติจากการมี `argv[3]` หรือไม่

ช่องทางรัน	ชื่อ job มาจาก	input มาจาก	ตัวอย่าง
Local / CLI / runbook	env `JOB_NAME`	env `INPUT` (JSON string)	`JOB_NAME=sgi-notify-pending-work INPUT='{"asOfDate":"2026-06-15","windows":[[30,36],[45,51],[60,66]]}' npm run start`
AWS Batch (ตารางเวลาจริง)	`process.argv[3]`	`process.argv[2]` (JSON string)	`node dist/main.js '{"asOfDate":"2026-06-15","windows":[[30,36],[45,51],[60,66]]}' sgi-notify-pending-work`

ตารางเวลาไม่ได้อยู่ในโค้ด — repo นี้ไม่มี `@Cron`/`@Interval` แม้แต่จุดเดียว (แม้ติดตั้ง `@nestjs/schedule` ไว้) cron ในหัวข้อ 5 เป็น นิยามของ AWS Batch scheduled event ที่ต้องตั้งตอน deploy ไม่ใช่ค่าที่อ่านจาก config file

Argument ที่รับได้ (`INPUT` เป็น JSON object · ไม่ส่ง = `{}`)

ระบบเดิมรับ argument แบบนี้: `SendMailReport.main` ไม่รับ args — เรียก `sendEmail(...)` 3 ครั้งตายตัวด้วย period `"30"`, `"45"`, `"60"`
ของใหม่เปลี่ยนจาก positional string เป็น JSON เพื่อให้เพิ่มฟิลด์ได้โดยไม่พังของเดิม แต่
ต้องรองรับทุกอย่างที่ระบบเดิมรับได้เป็นอย่างน้อย — ห้ามลดความสามารถลง

Field	ชนิด	ไม่ส่งแล้วได้อะไร (default)	Validation	ตรงกับ argument เดิม
`asOfDate`	string	วันนี้	`YYYY-MM-DD` · ใช้คำนวณอายุงาน	ของใหม่ — เดิมใช้วันปัจจุบันเสมอ ทดสอบย้อนหลังไม่ได้
`windows`	number[][]	`[[30,36],[45,51],[60,66]]`	คู่ [เริ่ม,จบ] · ต้องไม่ทับกัน · จบ ≥ เริ่ม	ของใหม่ — เดิม hardcode ใน `FgiConstant`
`zones`	string[]	`null` = ทุกโซน	รหัสโซนต้องมีจริง	ของใหม่
`dryRun`	boolean	`false`	`true` = อ่าน/คำนวณครบแต่ไม่ commit และไม่ publish/ไม่ส่งไฟล์	ของกลาง ทุก job
`limit`	number	`null`	จำกัดจำนวนรายการที่ประมวลผลรอบนี้ (ใช้ตอน smoke test)	ของกลาง ทุก job

กติกาการ validate ที่ทุก job ต้องทำเหมือนกัน — parse `INPUT` ไม่สำเร็จ หรือฟิลด์ไม่ผ่าน validation ให้ log `BATCH_END` ด้วย `batchStatus: 'FAILED'` แล้ว `exit(1)` ก่อนแตะฐานข้อมูล (ห้าม fallback ไปค่า default เงียบ ๆ เมื่อผู้ใช้ตั้งใจส่งค่ามาแล้วผิด) · ฟิลด์ที่ไม่รู้จักให้ log warn แล้วข้าม ไม่ทำให้ job ล้ม

- `dryRun=true` ให้ประกอบเนื้อหาอีเมลครบแต่ ไม่ส่งจริงและไม่เขียน `email_sent` — ใช้ตรวจว่าคัดงานได้ตรงก่อนเปิดใช้

2.2 Input / Output

Stage	Contract for implementation
Input	งานค้างในขั้นรอดำเนินการของ workflow engine + อายุงานเป็นวัน
Progress	อ่านงานค้างจาก engine, คำนวณอายุงาน, คัดเฉพาะช่วง 30-36 / 45-51 / 60-66 วัน, จัดกลุ่มตามโซน, ส่งอีเมลถึง GM และ OPT ต่อโซน
Output	อีเมลเตือนงานค้าง 1 ฉบับต่อโซนต่อช่วงอายุ พร้อมแถวใน email_sent

2.3 ขั้นตอนการทำงาน

Step	Description
1	เริ่ม
2	อ่านงานค้างด้วย SQL อ่านอย่างเดียวบน sps_store.workflow_transaction (เฉพาะ state ที่รอคนกด · ไม่เรียก getPendingFlowByUser ของ lib (มติ 2026-09-09 — workflow lib ใช้กับ flow K2 เท่านั้น) · job ต้องการงานค้างทั้งระบบเพื่อคิดอายุ 30/45/60 วัน ไม่ใช่ inbox รายคน · ห้ามเขียนตารางของ engine)
3	คำนวณอายุงาน (วัน) จากเวลาที่เข้าขั้นนั้น
4	อายุอยู่ในช่วง 30-36 / 45-51 / 60-66? | No: ข้ามรายการนั้น (นอกช่วงไม่ส่ง)
5	จัดกลุ่มตามโซน แล้วประกอบเนื้อหาอีเมลต่อโซน
6	ส่งอีเมลผ่าน @gosoft-sbp/email-lib ถึง GM (38) + OPT (15) (lib เขียน email_sent ให้เอง)
7	จบ

2.4 เงื่อนไขตัดสิน (Decision Rules) — ตัดสินจากอะไร

Job 12 ตัดสินเรื่องเดียวแต่พลาดง่ายที่สุดในชุด: งานค้างกี่วันถึงจะเตือน — ระบบเดิมใช้ "ช่วง 7 วัน" ไม่ใช่ "ครบ n วันขึ้นไป" ซึ่งเอกสารรุ่นก่อนเขียนผิดมาตลอด

คำถามที่โค้ดต้องตอบ	ตัดสินจาก (ตาราง · คอลัมน์)	เงื่อนไขที่ต้องเป็นจริง	ไม่เข้าเงื่อนไขแล้วทำอะไร
งานค้างชิ้นนี้ต้องเตือนหรือไม่	อายุงาน (วัน) นับจากเวลาที่เอกสารเข้าขั้นปัจจุบัน — `sps_store.workflow_transaction`	อายุต้องอยู่ใน ช่วงใดช่วงหนึ่งของ 3 ช่วงนี้: `30-36` · `45-51` · `60-66` วัน · นอกช่วงไม่ส่งเลย	▲ ค้าง 37-44 · 52-59 · หรือ 67 วันขึ้นไป จะเงียบสนิท — เป็นพฤติกรรมเดิม (`MailReportService.seperateDay` บรรทัด 177-183) ที่ผูกกับการรันสัปดาห์ละครั้ง พอดี · เปลี่ยนความถี่การรันเมื่อไร ต้องทบทวนช่วงพร้อมกัน ไม่งั้นงานจะหลุดการเตือน
งานแบบไหนถึงนับ	`sps_store.workflow_transaction.current_state_id` เทียบกับชุด state ที่รอคนกด	เฉพาะเอกสารที่อยู่ใน ขั้นรอดำเนินการ (ระบบเดิมคือ `stepID = 501001`) — ระบบใหม่ต้อง map เป็นชุด state ของ engine กลางแล้วเก็บใน config	เอกสารที่จบแล้ว (99) หรืออยู่ระหว่างระบบทำงานเอง ไม่นับ
ส่งหาใคร	โซนของร้านที่ถูกกระทบ (`mas_store.region`) + `business_user` ของระบบ SBP เดิม	จัดกลุ่มตามโซนก่อน แล้วส่ง 1 ฉบับต่อโซนต่อช่วงอายุ ถึง GM group 38 และ OPT group 15	โซนที่ไม่มีผู้รับ = log warn แล้วข้ามโซนนั้น ห้ามล้มทั้ง job

▲ ช่องว่างของ schema ที่ต้องปิดก่อน implement เงื่อนไขข้างบนได้จริง

แถวในตารางนี้ไม่ใช่ "ข้อควรระวัง" แต่เป็น ของที่ยังไม่มีในโครง 21 ตาราง — เขียนโค้ดตามเงื่อนไขด้านบนแล้วจะ compile ไม่ผ่าน/คิวรีพังทันที

#	สิ่งที่ขาด	ต้องทำอะไรก่อน
G7 ⧗ ต้องยืนยัน	ระบบเดิมใส่ วันที่ พ.ศ. ในอีเมล (`getCurrentBuddhistDate()`) ซึ่งขัดกับกติกา ค.ศ. ทั้งระบบ (มติ 2026-08-06)	ตัดสินว่าอีเมลจะคง พ.ศ. ตามของเดิม หรือเปลี่ยนเป็น ค.ศ. ให้ตรงทั้งระบบ — กระทบ template ที่ต้องเตรียม
G8 ⧗ ต้องยืนยัน	ไม่มี marker กันส่งซ้ำเมื่อ rerun วันเดียวกัน (ต่างจาก Job 10 ที่มี `last_ack_notified_on`)	ถ้าต้องกันซ้ำ ต้องเพิ่มคอลัมน์/ตาราง marker · ถ้ายอมให้ส่งซ้ำได้ตอน rerun ให้ระบุไว้ใน runbook

ค่าคงที่และโดเมนที่ใช้ในเงื่อนไขข้างบน

ทุกค่าในตารางนี้ต้องอ่านจาก config/env ไม่ hardcode ในคิวรี — แต่ ค่าตั้งต้นต้องเท่าระบบเดิมทุกตัว ไม่งั้นผลลัพธ์จะไม่ตรงกันตอนเทียบ UAT

ค่า	โดเมน / ค่าที่ระบบเดิมใช้	ที่มา
ช่วงอายุงาน	`30-36` · `45-51` · `60-66` วัน (ช่วงละ 7 วัน)	`FgiConstant.THIRTYDAY/THIRTYSIX/FOURTYFIVEDAY/FIFTYONE/SIXTYDAY/SIXTYSIX`
กลุ่มผู้รับ	GM = `38` · OPT = `15`	`FgiConstant.GM_GROUP_ID` / `OPT_GROUP_ID`
ขั้นที่นับว่าค้าง	ระบบเดิม `stepID = 501001`	`FgiConstant.STEPIDWAIT` — ระบบใหม่ map เป็น state ของ `@srm/glb-workflow`
จังหวะรัน	ทุกวันจันทร์ 10:00 น.	`Workflow design` · ต้องสอดคล้องกับช่วง 7 วันข้างบน

2.5 Field, Format, and Validation

Field / UI	Format	Validation	Behavior
กำหนดการรัน (Cron)	0 10 * * 1	แก้ไขได้	ทุกวันจันทร์ 10:00 น. ตาม Workflow design
ช่วงอายุงานที่เตือน	30-36 | 45-51 | 60-66 วัน	แก้ไขได้	เป็นช่วง 7 วัน ไม่ใช่ ">= n วัน" — นอกช่วงไม่ส่งเลย (พฤติกรรมเดิมของ MailReportService.seperateDay)
กลุ่มผู้รับ	GM group 38 · OPT group 15	แก้ไขได้	business_user ของระบบ SBP เดิม
การจัดกลุ่ม	ตามโซนของร้านที่ถูกกระทบ (zone)	ค่าคงที่/แก้ผ่านหน้าจอไม่ได้	1 อีเมลต่อ 1 โซนต่อ 1 ช่วงอายุ
Template	EM-04 (เตือนรายสัปดาห์) · EM-05 (escalation)	แก้ไขได้	ใช้ template ที่มีอยู่แล้วในชุด 8 ฉบับ — ไม่สร้าง EM ใหม่ · เลขจริงอ่านจาก mas_param · ระบบเดิมใช้ template 38 (EMAILTOSBP)

2.6 หลักฐานที่ต้องเก็บตอนรัน

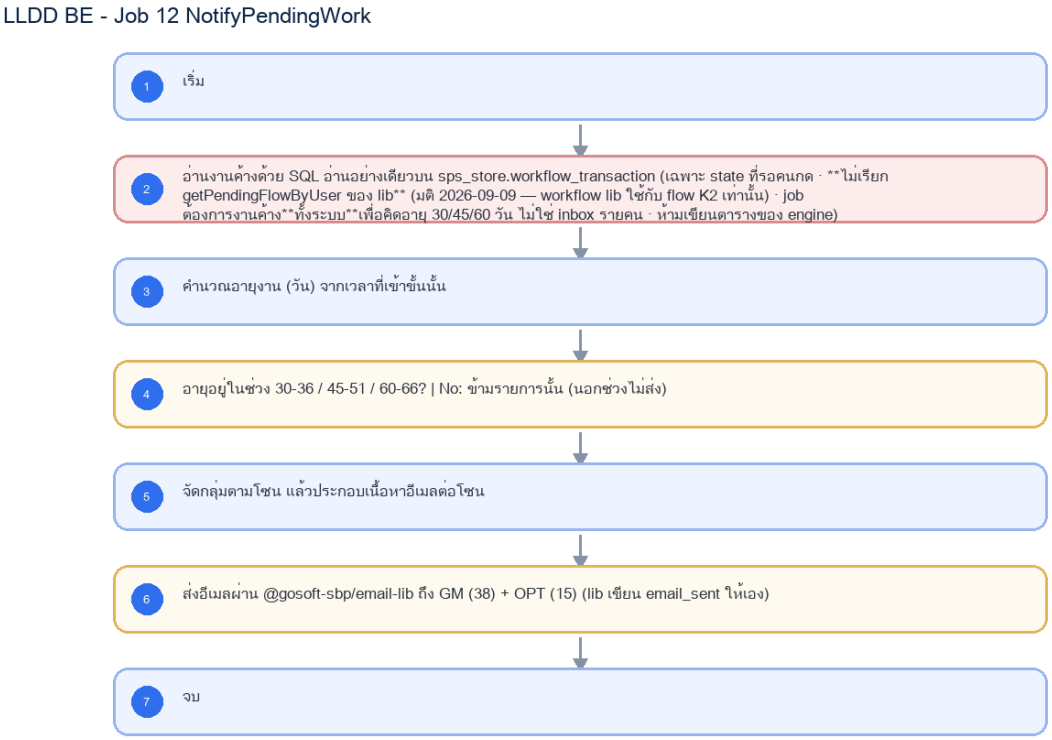
Evidence	Job-specific value	Acceptance
Input identity	งานค้างในขั้นรอดำเนินการของ workflow engine + อายุงานเป็นวัน	บันทึกไฟล์/คีย์ธุรกิจ/งวดที่รับเข้าใน run record
Output identity	อีเมลเตือนงานค้าง 1 ฉบับต่อโซนต่อช่วงอายุ พร้อมแถวใน email_sent	กระทบยอด input · สำเร็จ · ตก · ข้าม
Dedup proof	ส่งสัปดาห์ละครั้งตาม cron; รันซ้ำวันเดียวกันจะส่งซ้ำ — ถ้าต้องกันซ้ำให้ใช้ marker เดียวกับ Job 10 (last_notified_on) ซึ่งยังไม่มีในโครง (ดูช่องว่าง)	รันซ้ำด้วย fixture เดิมต้องไม่เกิดคีย์ธุรกิจซ้ำ
Transaction proof	อ่านอย่างเดียว ไม่มี DB transaction; ส่งเมลล้มเหลวรายโซนต้องไม่ทำให้โซนอื่นไม่ได้รับ (รวบ error แล้วรายงานท้ายรอบ)	จำลอง error กลางทางแล้วไม่เหลือแถวค้างนอกขอบเขตที่ระบุ
Security proof	อ่าน business_user ของระบบเดิมได้เฉพาะคอลัมน์อีเมล/กลุ่ม; ห้าม log อีเมลผู้รับแบบเต็มใน application log	config/log/error ไม่มีความลับแบบข้อความธรรมดา

2.7 โค้ด Java เดิมที่อ้างอิง

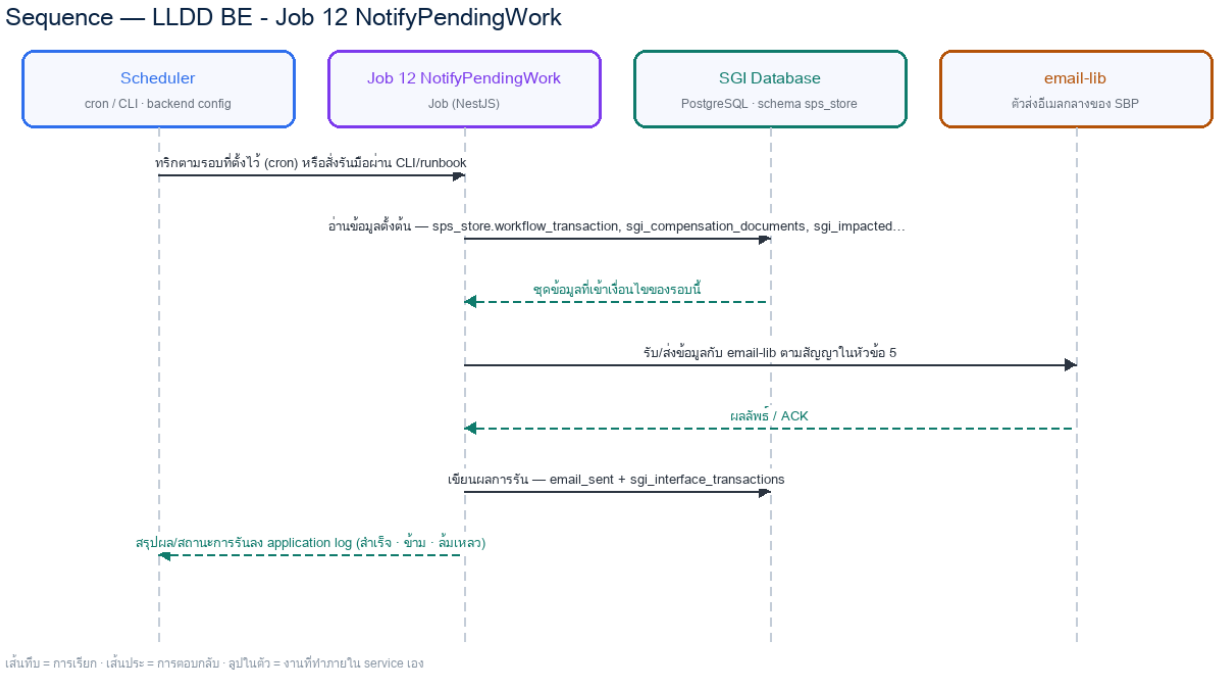
Legacy file	Line range	Responsibility to carry forward
fcsJar/src/th/co/gosoft/fgi/main/SendMailReport.java	23-60	Legacy main entrypoint — เรียก 3 ช่วง 30/45/60 ตามลำดับ
fcsJar/src/th/co/gosoft/fgi/service/MailReportService.java	168-240	seperateDay() คัดช่วงอายุงาน + sendEmail() จัดกลุ่มตามโซนและ resolve ผู้รับ GM/OPT

Line ranges refer to the legacy Java implementation under `batchjob/fcsJar/` (path นับจากราก `sbp-prototype/`). Use these ranges to preserve business behavior while implementing the target Node job.

3. Flow และ Sequence Diagram



รูปที่ 1: Implementation flow: LLDD BE - Job 12 NotifyPendingWork



รูปที่ 2: Sequence diagram: LLDD BE - Job 12 NotifyPendingWork

4. Reference DB Mapping

อ้างอิงสำหรับ implement job เท่านั้น ไม่ใช่งานออกแบบ/สร้างหน้า Database

Table / Object	R/W	Usage
sps_store.workflow_transaction	R	งานค้างของ engine กลาง — ห้ามเขียน
sgi_compensation_documents	R	เลขเอกสาร · ร้าน · งวด สำหรับเนื้อหาอีเมล
sgi_impacted_stores	R	โซนของร้านที่ถูกกระทบ ใช้จัดกลุ่มผู้รับ
business_user (ระบบ SBP เดิม)	R	อีเมลของ GM group 38 และ OPT group 15
email_sent (ระบบ SBP เดิม)	W (โดย @gosoft-sbp/email-lib)	lib เขียน log ให้เอง

5. โค้ดจริงอยู่ที่ไหน

เอกสารนี้ ไม่วาง skeleton code — job ทั้ง 11 ตัว implement แล้ว การมีโค้ดสองชุด (ในเอกสาร + ของจริง) ทำให้หลุดจากกันแน่นอน · อ่านสัญญาที่หัวข้อ 2 แล้วดูของจริงตามตารางนี้

สิ่งที่ต้องดู	ที่อยู่
ชื่อ job ที่ลงทะเบียน	`sgi-job-12` ใน `SBP/srm-sps-spsap-sop-sgi-batch/src/main.ts`
service ของ job	`SBP/srm-sps-spsap-sop-sgi-batch/src/modules/sgi/job-12-*.service.ts`
เทสของ job	`SBP/srm-sps-spsap-sop-sgi-batch/src/modules/sgi/job-12-*.spec.ts`
SQL ที่ใช้จริง	อยู่ในไฟล์ service เดียวกัน (repo นี้ไม่แยก `*.sql.ts`)
คำอธิบายภาษาคนของ job	batchjob/JOB-12-* อธิบายละเอียด

6. Acceptance Criteria

- พารามิเตอร์รับผ่าน `INPUT` (JSON) และ config ของ repo — เปลี่ยนค่าโดย deploy ไม่ใช่ผ่าน API/หน้าจอ · ตารางเวลาตั้งที่ AWS Batch scheduled event ไม่ใช่ `@Cron` ในโค้ด
- การรันต้องตรวจ enabled flag ใน config · การกันรันซ้อนพึ่ง AWS Batch queue เป็นหลัก — advisory lock เป็นของใหม่ที่ต้องสร้างเองถ้างานนั้นรับความเสี่ยงรันซ้อนไม่ได้
- ทุกรอบต้องเขียน application log แบบ structured (`BATCH_START`/`BATCH_END` + `runId` — `src/main.ts` ทำให้แล้ว) และบันทึกลง `integration_log` อัตโนมัติ · error ต้องส่ง EM-07
- DB/table mapping ใช้เป็น reference สำหรับ implement Job เท่านั้น ไม่ใช่งานสร้างหน้า Database
- รองรับ rerun rule และ risk note ตาม runbook

7. Unit Test Case

7 เคส · ทุกเคสต้องรันได้โดยไม่ต่อ DB/บริการภายนอกจริง (mock ที่ขอบ repository/client) · ข้อความไทยที่ยืนยันในเทสต้องเป็น verbatim ตาม ข้อกำหนดทางธุรกิจ ห้ามพิมพ์ใหม่ · ● เทสที่ mock query runner ต้องป้อนรูป tuple `[rows, rowCount]` สำหรับ UPDATE/DELETE (กฎ #140)

ช่วงอายุงาน

Test ID	เงื่อนไข	Input	ผลลัพธ์ที่ควรได้
UT-01	อยู่ในช่วงที่ต้องเตือน	อายุงาน 33 วัน	เตือน (ช่วง 30-36) — ช่วงมาจาก `FgiConstant.THIRTYDAY/THIRTYSIX`
UT-02	อยู่นอกทุกช่วง	อายุงาน 40 วัน	ไม่เตือน (อยู่ระหว่าง 36 กับ 45)
UT-03	ครบทั้งสามช่วง	อายุ 33 · 48 · 63 วัน	เตือนทั้งสามช่วง (30-36 · 45-51 · 60-66)

งานที่นับ

Test ID	เงื่อนไข	Input	ผลลัพธ์ที่ควรได้
UT-04	เอกสารอยู่ขั้นรอดำเนินการ	`current_state_id` อยู่ในชุด state ที่รอคนกด	นับ
UT-05	เอกสารจบแล้ว	`current_state_id` เป็น state ปลายทาง	ไม่นับ

ผู้รับอีเมล

Test ID	เงื่อนไข	Input	ผลลัพธ์ที่ควรได้
UT-06	จัดกลุ่มตามโซน	งานค้าง 5 ชิ้นใน 2 โซน ช่วงอายุเดียวกัน	ส่ง 1 ฉบับต่อโซนต่อช่วงอายุ (= 2 ฉบับ) ถึง GM group 38 และ OPT group 15
UT-07	โซนที่หาผู้รับไม่ได้	โซนไม่มีสมาชิกใน group	ข้ามโซนนั้นแล้ว log ไว้ · ไม่ทำให้ job ล้มทั้งรอบ

ภาคผนวก: Java Source เดิม

Code ในส่วนนี้คัดจาก Java source เดิมโดยตรงและคงข้อความตามต้นฉบับ ใช้เลขบรรทัดที่ระบุหน้าทุก snippet เพื่อตรวจเทียบ business behavior, SQL, transaction และ error handling

J1. SendMailReport.java — lines 23-34

รายการ	รายละเอียด
Source file	fcsJar/src/th/co/gosoft/fgi/main/SendMailReport.java
Original lines	23-34
Responsibility	Legacy main entrypoint — เรียก 3 ช่วง 30/45/60 ตามลำดับ

```
public static void main(String[] args) {
LogUtils.info(SendMailReport.class,"====*** Start Job SendMailReport***===");
try {

JsonArray jsonArray = new JsonParser().parse(mailReportService.getImpactStoreJson()).getAsJsonArray();
if(jsonArray.size()==0){
LogUtils.info(SendMailReport.class,"No data from K2");
}else{
List<ImpactProcess> impactStoreProcess = mailReportService.getImpactStore(jsonArray);
HashMap<Integer, List<ImpactProcess>> impactstorePeriod= mailReportService.seperateDay(impactStoreProcess);

if(mailReportService.sendEmail(impactstorePeriod.get(FgiConstant.THIRTYDAY),"30")){
```

J1. SendMailReport.java — lines 49-60

รายการ	รายละเอียด
Source file	fcsJar/src/th/co/gosoft/fgi/main/SendMailReport.java
Original lines	49-60
Responsibility	Legacy main entrypoint — เรียก 3 ช่วง 30/45/60 ตามลำดับ

```
LogUtils.info(SendMailReport.class,"===***End Job Send Email ***===");
}

} catch (Exception e) {
LogUtils.error(SendMailReport.class,e);
LogUtils.info(SendMailReport.class,"===***Job Send Email Failed***===");
}

}
```

J2. MailReportService.java — lines 168-179

รายการ	รายละเอียด
Source file	fcsJar/src/th/co/gosoft/fgi/service/MailReportService.java
Original lines	168-179
Responsibility	seperateDay() คัดช่วงอายุงาน + sendEmail() จัดกลุ่มตามโซนและ resolve ผู้รับ GM/OPT

```
public HashMap seperateDay(List<ImpactProcess> impactProcess){
HashMap<Integer,List<ImpactProcess>> mapPeriod = new HashMap<Integer, List<ImpactProcess>>();
List<ImpactProcess> dayProcess30 = new ArrayList<ImpactProcess>();
List<ImpactProcess> dayProcess60 = new ArrayList<ImpactProcess>();
List<ImpactProcess> dayProcess45 = new ArrayList<ImpactProcess>();
int waitDay;
for(int index=0;index<impactProcess.size();index++){
if(FgiConstant.STEPIDWAIT.equalsIgnoreCase(impactProcess.get(index).getStepID())){
waitDay = Integer.valueOf(impactProcess.get(index).getWaitDay());
if(waitDay>=FgiConstant.THIRTYDAY && waitDay<=FgiConstant.THIRTYSIX){
dayProcess30.add(impactProcess.get(index));
```

J2. MailReportService.java — lines 229-240

รายการ	รายละเอียด
Source file	fcsJar/src/th/co/gosoft/fgi/service/MailReportService.java
Original lines	229-240
Responsibility	seperateDay() คัดช่วงอายุงาน + sendEmail() จัดกลุ่มตามโซนและ resolve ผู้รับ GM/OPT

```
for(int index=0;index<regionList.size();index++){
sendList = mapImpactProcess.get(regionList.get(index));
email = regionGroup.get(regionList.get(index));
if(sendMailHTTP(email.get(FgiConstant.GM_GROUP_ID_INT),email.get(FgiConstant.OPT_GROUP_ID_INT),regionList.get(index),period,DateUtils.getCurrentBuddhi
stDate(),sendList)){
status = true;
LogUtils.info(getClass(),"Send Email to: "+email.get(FgiConstant.GM_GROUP_ID_INT)+" Zone: "+regionList.get(index)+" peroid: "+period);
}else{
check = false;
LogUtils.info(getClass(),"Cann't send Email to: "+email.get(FgiConstant.GM_GROUP_ID)+" Zone: "+regionList.get(index)+" peroid: "+period);
}
}
```